UNIVERSITY OF SCIENCE, VNU-HCM

FACULTY OF INFORMATION TECHNOLOGY
ADVANCED PROGRAM IN COMPUTER SCIENCE

CS486 - Introduction to Database System

Phase 2 Report

Campus Space Management System

Students:
Tran Canh Anh Tuan 24125048
Le Dat Phuc An 24125052
Vong Chi Van 24125049
Pham Nguyen Minh Quan 24125041

Lecturers:
Nguyen Ngoc Toan, MSc.
Nguyen Ngoc Minh Chau, MSc.
Le Thi Nhan, PhD

2026

Summary

§1 Introduction

§1.1 Background and Phase 2 Objectives

Phase 2 extends the Campus Space Management System after a one-semester pilot. The original database treated every active maintenance record as a complete booking block. The Phase 2 requirements distinguish disruptive maintenance from advisory work, introduce automatic approval for selected role–space-type combinations, require correctness under concurrent approval, and add analytical reporting over a substantially larger history. The implementation therefore has five objectives:

- represent advisory and out-of-service maintenance separately;
- preserve proof that requesters were notified about active advisories;
- prevent overlapping approvals even when automatic and staff workflows run concurrently;
- support four required operational and analytical reports; and
- demonstrate measurable indexing improvements on at least 100,000 bookings.

§1.2 Group Members and Contributions

Member	Student ID	Phase 2 contributions
Tran Canh Anh Tuan	24125048	Shared Phase 2 ERD and logical design (09); requirement-change analysis and concurrency design, implementation, and tests (08, 11–13).
Le Dat Phuc An	24125052	Shared Phase 2 ERD and logical design (09); schema migration, data generation, and analytical queries (10, 14, 16).
Vong Chi Van	24125049	Shared Phase 2 ERD and logical design (09); requirement-change analysis and concurrency design, implementation, and tests (08, 11–13).
Pham Nguyen Minh Quan	24125041	Shared Phase 2 ERD and logical design (09); index tuning and analytical-query refinement (15, 16).

Table 1: Group membership and deliverable ownership.

§1.3 Phase 2 Deliverables

Artifact	Purpose
08	Requirement-change and concurrency-conflict analysis
09	Phase 2 ERD and logical schema
10	Data-preserving migration from the Phase 1 schema
triggers.sql	Four operational triggers for booking, maintenance, schedule, and usage rules
11--13	Concurrency design, protected procedure, and two-session tests
14	Deterministic three-academic-year data generator
15	Indexing experiment and before/after measurements
16	Four required analytical queries considered in this report

Table 2: Phase 2 artifact summary.

This report is based on the submitted artifacts and the captured SQL Server test evidence. SQL excerpts are intentionally shortened; the complete executable statements are stored in outputs/, including outputs/triggers.sql.

§2 Requirement Changes and Database Evolution

§2.1 Maintenance Impact Levels

The Phase 2 rule separates maintenance into two impact levels:

out_of_service
: The space is unusable. An active record blocks every booking whose half-open interval overlaps the maintenance interval.

advisory
: The space remains bookable. Every active advisory at booking time must be communicated to the requester, and the database must retain evidence of that communication.

Several active records can coexist for one space. An open record can also be escalated or downgraded. Escalation to out_of_service does not erase earlier approvals; instead, staff must be able to retrieve the affected bookings and contact their requesters.

§2.2 Affected Concepts and Rules

Concept	Phase 2 change
MAINTENANCE_RECORD	Adds impact_level; overlapping active out-of-service work blocks approval, while advisory work permits approval and requires notification.
BOOKING_NOTIFICATION	Associates a booking and maintenance record with a notification type and timestamp, preserving advisory and escalation history.
BOOKING_DECISION	Replaces the approval-only relation and represents automatic and staff decisions uniformly.
ROLE, SPACE_TYPE	Normalize policy dimensions that were stored as text in Phase 1.
AUTO_USAGE_POLICY	Defines which role and space-type combinations are eligible for automatic processing.
Approval workflow	Every approval path must use the same atomic availability-check-and-write protocol.

Table 3: Principal requirement-driven design changes.

§2.3 Phase 2 Conceptual Overview

The Phase 2 design contains eleven relations. Figure 1 emphasizes foreign-key dependencies and the common data flow from requests to decisions, sessions, maintenance notifications, and reports.

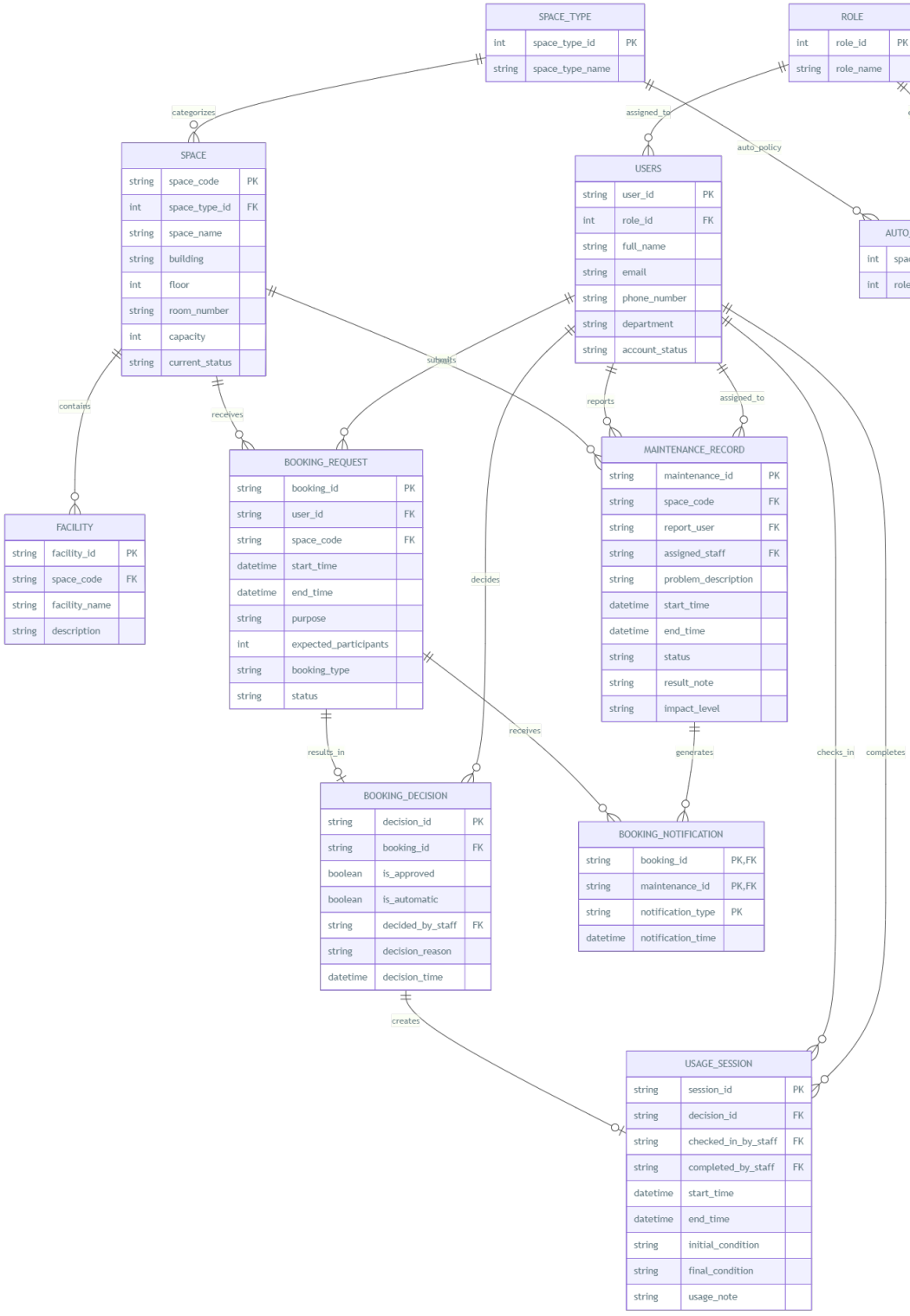

Figure 1: Phase 2 Schema Overview

Relation	Primary key	Important foreign/candidate keys
ROLE	role_id	role_name is unique
USERS	user_id	role_id; email is unique
SPACE_TYPE	space_type_id	space_type_name is unique
SPACES	space_code	space_type_id; (building, room_number) is unique
AUTO_USAGE_POLICY	(space_type_id, role_id)	both columns are foreign keys
FACILITY	facility_id	space_code
BOOKING_REQUEST	booking_id	user_id, space_code
BOOKING_DECISION	decision_id	booking_id is unique; decided_by_staff
USAGE_SESSION	session_id	decision_id is unique; two staff references
MAINTENANCE_RECORD	maintenance_id	space_code and three user-role references
BOOKING_NOTIFICATION	(booking_id, maintenance_id, notification_type)	booking_id and maintenance_id

Table 4: Phase 2 relation and key summary.

§2.4 Schema Migration

Output 10 applies the changes on top of the Phase 1 database rather than rebuilding it. Textual roles and space types are first copied into lookup tables; the original rows then receive foreign-key identifiers. Approval records are transformed into booking decisions before the old approval table is removed. Usage sessions are reconnected through their decisions, and new policy and notification relations begin empty because Phase 1 contained no equivalent facts.

Migration operation	Preservation approach
Normalize roles and space types	Insert distinct Phase 1 values, populate new identifiers by joins, enforce non-null foreign keys, then remove text columns.
Replace booking approval	Insert one BOOKING_DECISION for each existing approval, preserving its identifier, booking, staff member, reason, and time.
Reconnect usage sessions	Resolve each session's booking to the migrated decision, enforce a unique decision reference, and remove the former booking reference.
Extend maintenance	Add impact_level with out_of_service as the migration default, preserving the stricter Phase 1 behavior.
Add new concepts	Create empty automatic-policy and booking-notification relations because no reliable Phase 1 facts exist to infer.
Verify	Finish with table row counts; expected migrated Phase 1 counts include 20 users, 12 spaces, 30 requests, 25 decisions, 16 sessions, and 12 maintenance records.

Table 5: Data-preserving migration strategy.

§2.5 Operational Trigger Layer

The standalone outputs/triggers.sql script defines four operational triggers without changing the eleven-relation schema. New-request processing locks the existing SPACES row while checking maintenance, and its automatic-approval branch calls the concurrent-safe approval procedure.

Trigger	Event / errors	Responsibility
`trg_G08_ProcessNewBooking`	Request insert; 52301	Locks the requested space; rejects an overlapping out-of-service request; records active advisory notifications; and sends policy-eligible requests to the concurrent-safe automatic approval procedure.
`trg_G08_MaintenanceBookingNotifications`	Maintenance insert/update	Records `ADVISORY` or `OUT_OF_SERVICE` notifications for approved overlapping bookings when maintenance is created, changed, or escalated. The composite notification key and `NOT EXISTS` check prevent duplicate event types.
`trg_G08_ProtectDecidedBookingSchedule`	Request update; 52305	Prevents direct changes to the space or requested interval after a booking has a decision; status-only updates remain allowed.
`trg_G08_UsageSessionRequiresApproval`	Session insert/update; 52306	Requires every usage session to reference a decision whose `is_approved` value is true.

Table 6: Operational triggers defined in `outputs/triggers.sql`.

The trigger set covers both directions of maintenance interaction. A booking request is checked against existing maintenance, while an inserted or changed maintenance record identifies approved overlapping bookings and preserves a notification for staff follow-up. Schedule immutability prevents a decided booking from changing through an unchecked time or space edit, and usage-session validation requires an approved decision.

§2.6 Integrity Boundary

Primary keys, foreign keys, uniqueness, and check constraints enforce structural rules such as valid identifiers and time order. The concurrent-safe procedure enforces approval availability and interval overlap. Transactional triggers handle maintenance-aware request processing, automatic-policy dispatch, booking notifications, decided-schedule immutability, and usage eligibility.

§3 Concurrency Design and Test Results

§3.1 Identified Conflict

The critical anomaly is a check-then-act race. Consider two pending requests for the same space:

$$A = [09{:}00, 11{:}00), \qquad B = [10{:}00, 12{:}00).$$

Without a shared serialization resource, Session A and Session B can both read *no approved overlap* before either transaction inserts its decision. The unique constraint on `BOOKING_DECISION.booking_id` cannot prevent this outcome because the sessions decide two different booking identifiers.

Step	Session A	Session B
1	Checks overlap; none exists	
2		Checks overlap; none exists
3	Inserts approval A	
4		Inserts approval B
5	Commits	Commits

Table 7: Unsafe concurrent check-and-write schedule.

The required invariant uses half-open intervals: two approved, non-cancelled bookings for one `space_code` must never satisfy

$$existing.start < candidate.end \quad \wedge \quad candidate.start < existing.end.$$

§3.2 Protected Approval Protocol

The protected approval script selects the target database before creating the procedure. The procedure uses pessimistic SQL Server locking in one transaction:

1. Read the target `BOOKING_REQUEST` with `UPDLOCK`. This keeps its space, interval, and pending status stable.
2. Lock the existing `SPACES` row for that `space_code` with `UPDLOCK`.
3. Check approved, non-cancelled overlapping bookings.
4. Insert the decision and update the request to `approved`.
5. Commit, releasing both update locks.

Listing 1: Core locking order in the protected approval procedure.

```
BEGIN TRANSACTION;

SELECT @space_code = br.space_code,
       @start_time = br.start_time,
       @end_time = br.end_time,
       @status = br.status
FROM dbo.BOOKING_REQUEST AS br WITH (UPDLOCK)
WHERE br.booking_id = @booking_id;

SELECT @space_lock_code = s.space_code
FROM dbo.SPACES AS s WITH (UPDLOCK)
WHERE s.space_code = @space_code;

-- overlap check, decision insert, and status update
COMMIT TRANSACTION;
```

Competing requests for the same space therefore request incompatible update locks on one common row. Requests for different spaces lock different rows and can still proceed concurrently. The fixed procedure order `BOOKING_REQUEST` then `SPACES` reduces deadlock risk.

§3.3 Unsafe Test

The unsafe scripts deliberately bypass the protected procedure. Session A pauses after its availability check, Session B performs the same check and commits, and Session A then commits its stale decision. Figure 2 records the resulting violation.

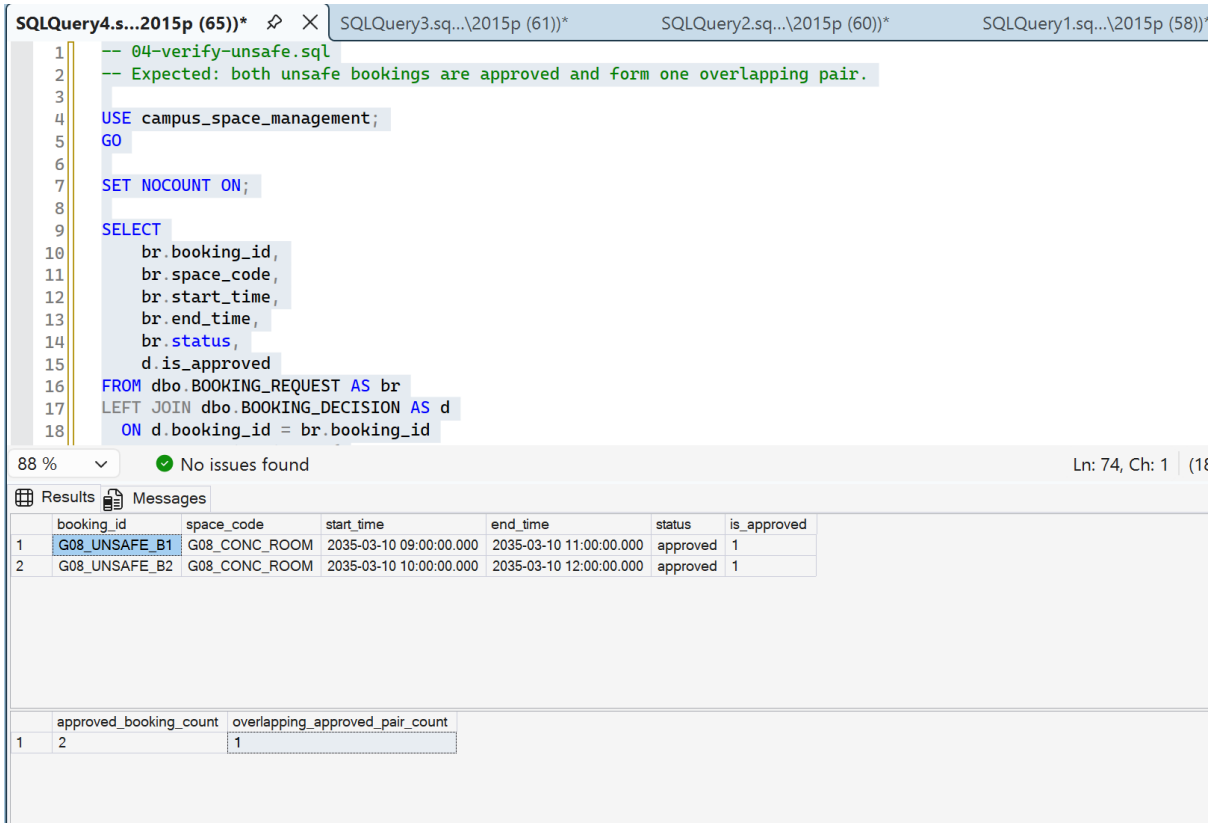

Figure 2: Unsafe verification: both requests are approved and one overlapping approved pair exists.

The verification reports

$$approved\_booking\_count = 2, \qquad overlapping\_approved\_pair\_count = 1.$$

§3.4 Protected Test

In the protected run, Session A acquires both update locks and holds them for twelve seconds. Figure 3 shows its successful decision.

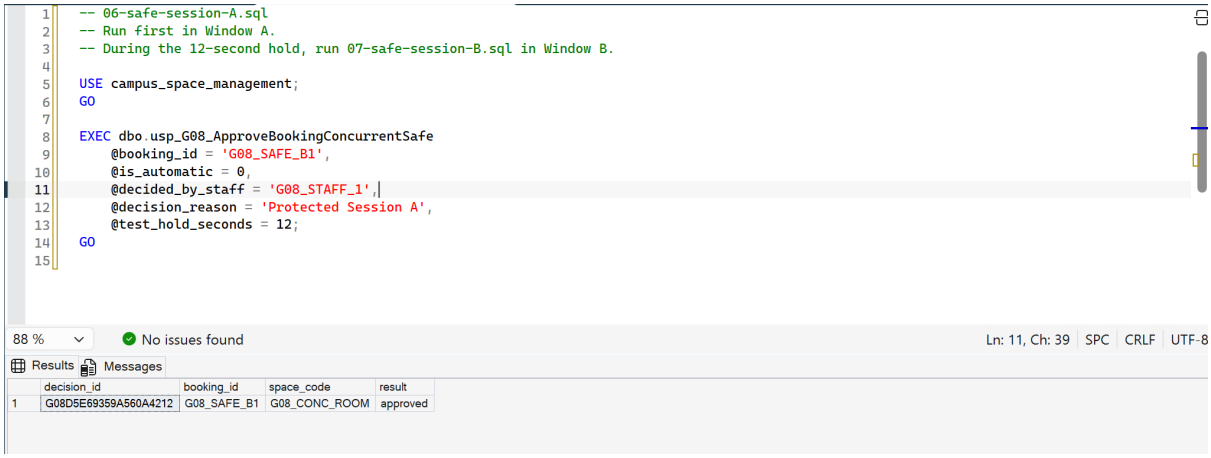

Figure 3: Protected Session A approves the first request while holding the booking and space locks.

Session B can lock its distinct request row but must wait for the same `SPACES` row. After Session A commits, Session B rechecks the database, detects the committed overlap, and receives application error 52105, as shown in Figure 4.

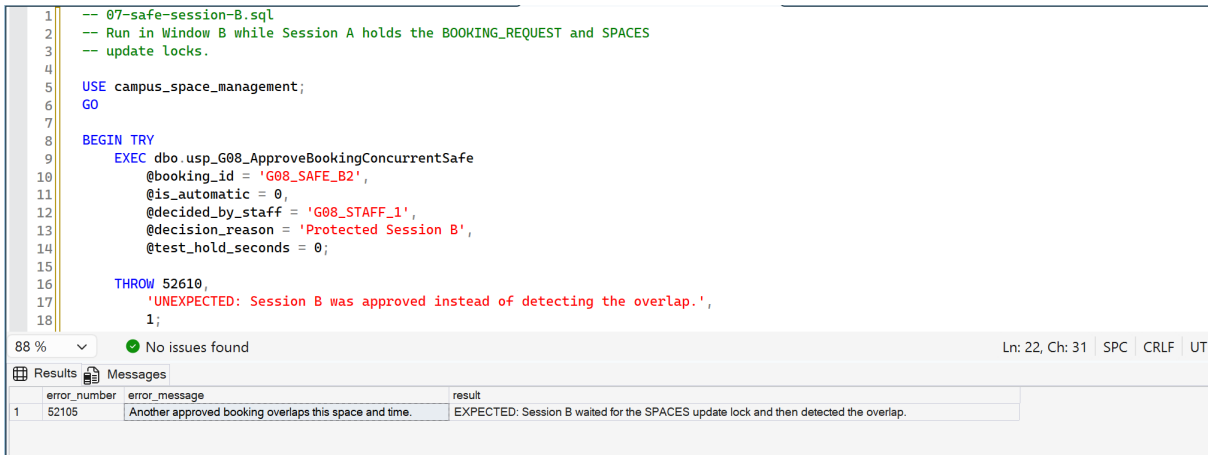

Figure 4: Protected Session B receives the expected overlap error after waiting for the space lock.

The final assertion in Figure 5 confirms that the first request is approved, the second remains pending, and no overlapping approved pair exists.

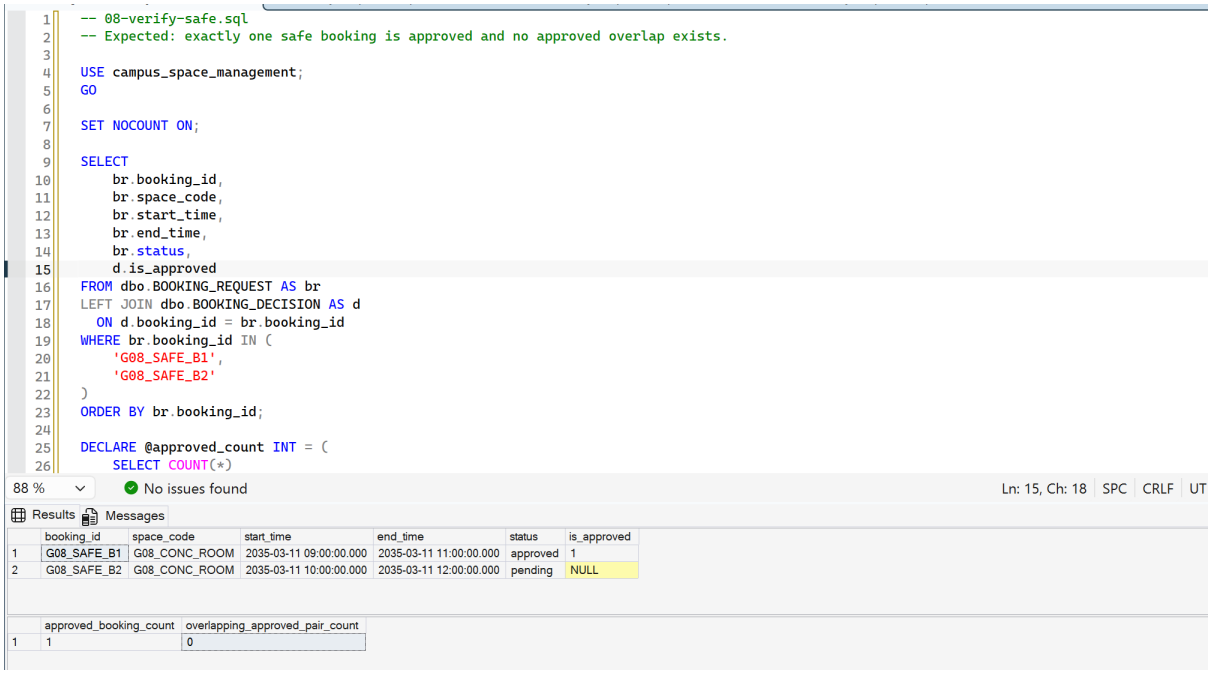

Figure 5: Final protected-state verification.

Test	Approved bookings	Overlapping approved pairs
Unsafe workflow	2	1
Protected workflow	1	0

Table 8: Observed concurrency-test results.

The test demonstrates both sides of the claim: the unprotected check can violate the invariant, while the common per-space update lock serializes the check-and-write section and prevents the violation.

§4 Generated Data and Required Analytical Queries

§4.1 Data Generation

Output 14 is a deterministic and rerunnable Microsoft SQL Server generator. It creates exactly 100,000 booking requests in a reserved identifier namespace while preserving migrated Phase 1 rows. The generated period is 1 September 2023 through 31 August 2026, covering three complete academic years.

Coverage dimension	Generated evidence
Scale	100,000 generated requests; the tuning database contains 100,030 requests after retaining Phase 1 data.
Users and spaces	2,000 requesters, three processing/staff accounts, and 50 generated spaces.
Booking lifecycle	Approved, rejected, pending, completed, cancelled, and no-show cases.
Processing modes	Automatic and staff decisions consistent with `AUTO_USAGE_POLICY`.
Maintenance	Advisory and out-of-service records, including overlapping active work and notification records.
Integrity checks	Assertions verify the row count, date span, lifecycle coverage, policy consistency, absence of approved overlaps, and advisory-notification completeness.

Table 9: Generated dataset characteristics.

The performance experiment reports 100,030 booking requests, 98,025 booking decisions, 782 maintenance records, 182 facilities, 62 spaces, and six space types. This scale is sufficient to make scan-versus-seek differences visible.

§4.2 Query 01: Approved Booking Hours per Space

Business question. What is the total approved booking time for every space during a supplied semester?
Users. Facility Manager.
The query joins requests to approved decisions, spaces, and space types. It excludes later cancellations, applies a half-open semester boundary, and aggregates duration in minutes before converting it to decimal hours.

Listing 2: Core of Query 01.

```
SELECT s.space_code, s.space_name, st.space_type_name,
       COUNT(*) AS approved_booking_count,
       CAST(SUM(DATEDIFF(MINUTE, br.start_time, br.end_time))
            / 60.0 AS DECIMAL(10,2)) AS total_approved_hours
FROM BOOKING_REQUEST AS br
JOIN BOOKING_DECISION AS bd ON bd.booking_id = br.booking_id
JOIN SPACES AS s ON s.space_code = br.space_code
JOIN SPACE_TYPE AS st ON st.space_type_id = s.space_type_id
WHERE bd.is_approved = 1
  AND br.status <> 'cancelled'
  AND br.start_time >= @SemesterStart
  AND br.start_time < @SemesterEnd
GROUP BY s.space_code, s.space_name, st.space_type_name;
```

§4.3 Query 02: Approved Bookings by Weekday and Hour

Business question. How are approved booking decisions distributed by weekday and hour during a supplied semester?
Users. Facility Manager and Department Administrator.

The implementation uses `BOOKING_DECISION.decision_time`, filters an optional hour interval, and groups by the SQL Server weekday name, weekday number, and hour. Cancelled bookings are excluded even if their historical decision was approved.

Listing 3: Core of Query 02.

```
SELECT DATENAME(WEEKDAY, bd.decision_time) AS weekday_name,
       DATEPART(WEEKDAY, bd.decision_time) AS weekday_number,
       DATEPART(HOUR, bd.decision_time) AS decision_hour,
       COUNT(*) AS booking_count
FROM BOOKING_DECISION AS bd
JOIN BOOKING_REQUEST AS br ON br.booking_id = bd.booking_id
WHERE bd.is_approved = 1
  AND br.status <> 'cancelled'
  AND bd.decision_time >= @SemesterStart
  AND bd.decision_time < @SemesterEnd
  AND DATEPART(HOUR, bd.decision_time)
      BETWEEN @FromHour AND @ToHour
GROUP BY DATENAME(WEEKDAY, bd.decision_time),
         DATEPART(WEEKDAY, bd.decision_time),
         DATEPART(HOUR, bd.decision_time);
```

§4.4 Query 03: Available-Space Finder

Business question. Which available spaces meet a required capacity and complete facility list during a requested time interval?
Users. Students, lecturers, teaching assistants, and facility staff.
A result must meet all five conditions: operational status, adequate capacity, every requested facility, no approved non-cancelled overlap, and no active out-of-service maintenance overlap. Advisory maintenance does not remove a result; it is returned as a warning flag.

Listing 4: Key predicates in Query 03.

```
WHERE s.current_status = 'available'
  AND s.capacity >= @MinCapacity
  AND @RequiredFacilityCount = (
      SELECT COUNT(DISTINCT f.facility_name)
      FROM FACILITY AS f
      JOIN @RequiredFacilities AS rf
        ON rf.facility_name = f.facility_name
      WHERE f.space_code = s.space_code
  )
  AND NOT EXISTS (
      SELECT 1
      FROM BOOKING_REQUEST AS br
      JOIN BOOKING_DECISION AS bd
        ON bd.booking_id = br.booking_id
      WHERE br.space_code = s.space_code
        AND bd.is_approved = 1
        AND br.status <> 'cancelled'
        AND br.start_time < @RequestedEnd
        AND br.end_time > @RequestedStart
  )
  AND NOT EXISTS (
      SELECT 1
      FROM MAINTENANCE_RECORD AS m
```

```
      WHERE m.space_code = s.space_code
        AND m.impact_level = 'out_of_service'
        AND m.status IN ('pending', 'in_progress')
        AND m.start_time < @RequestedEnd
        AND (m.end_time IS NULL
             OR m.end_time > @RequestedStart)
  );
```

§4.5 Query 04: Bookings Affected by Maintenance Escalation

Business question. Which previously approved bookings were affected when a selected advisory record was escalated to out-of-service?
Users. Facility Staff and Facility Manager.
The notification table supplies durable evidence of an actual escalation. The report selects `OUT_OF_SERVICE` notifications for the chosen maintenance identifier, requires an approved decision, and returns requester contact details, booking times, decision time, escalation time, and maintenance details.

Listing 5: Core of Query 04.

```
SELECT br.booking_id, u.full_name, u.email,
       br.space_code, br.start_time, br.end_time,
       br.status AS booking_status,
       bd.decision_time,
       bn.notification_time AS escalation_time,
       m.problem_description, m.impact_level
FROM BOOKING_NOTIFICATION AS bn
JOIN BOOKING_REQUEST AS br ON br.booking_id = bn.booking_id
JOIN BOOKING_DECISION AS bd ON bd.booking_id = br.booking_id
JOIN MAINTENANCE_RECORD AS m
  ON m.maintenance_id = bn.maintenance_id
JOIN USERS AS u ON u.user_id = br.user_id
WHERE bn.maintenance_id = @MaintenanceId
  AND bn.notification_type = 'OUT_OF_SERVICE'
  AND bd.is_approved = 1
ORDER BY bn.notification_time, br.start_time;
```

§4.6 Requirement Traceability

Query	Required report	Principal relations
01	Approved hours for each space	Requests, decisions, spaces, space types
02	Approved count by weekday and hour	Decisions and requests
03	Capacity/facility/time room finder	Spaces, facilities, requests, decisions, maintenance
04	Bookings affected by escalation	Notifications, maintenance, decisions, requests, users

Table 10: Traceability of the four required analytical queries.

§5 Indexing and Query Tuning

§5.1 Scope and Methodology

The tuning experiment evaluates four workloads: the booking-conflict check, the Query 03 room finder, Query 01, and Query 02. The same generated database and query semantics were used before and after indexing.

The database ran Microsoft SQL Server 2022 at compatibility level 160. Each phase removed or created only the experimental indexes, refreshed statistics with `FULLSCAN`, warmed the workload, recompiled benchmark procedures, and measured each target five times. The complete baseline and tuned sequence was repeated three times. Logical reads, CPU time, elapsed time, access operators, index usage, and storage were recorded.

The tuning folder contains a read-only index-inventory diagnostic. It lists index type, uniqueness, primary-key status, and unique-constraint status for the five workload tables, making the baseline access structures explicit before interpreting scan and seek behavior. The benchmark harness preserves the measured workloads and result methodology used below.

§5.2 Selected Indexes

Index	Workload rationale
`IX_G08_BR_Space_Start`	Keys `BOOKING_REQUEST` by `(space_code,start_time)` and includes end time/status for conflict and room-finder overlap checks.
`IX_G08_BR_Start`	Keys requests by `start_time` and covers space, end time, and status for Query 01 semester access and a narrower Query 02 scan.
`IX_G08_BD_Approved_Booking`	Filtered approved-only `booking_id` lookup for conflict checks and joins.
`IX_G08_BD_Approved_DecisionTime`	Filtered approved-only decision-time range with included booking identifier for Query 02.
`IX_G08_Facility_Space_Name`	Supports direct facility lookup by space and required facility name.
`IX_G08_Maint_Space_Impact_Status_Start`	Narrows maintenance by space, impact, active status, and interval start while covering the interval end.

Table 11: Final workload-specific index set.

§5.3 Before-and-After Results

Target	Logical reads		Reduction	Elapsed ms		Reduction
	Before	After		Before	After	
Conflict check	55.00	14.00	74.55%	0.31	0.08	73.12%
Room finder	35,152.20	953.40	97.29%	787.54	15.42	98.04%
Query 01	2,760.00	641.00	76.78%	42.31	28.73	32.10%
Query 02	2,677.00	829.00	69.03%	41.38	31.03	25.00%

Table 12: Average performance before and after indexing.

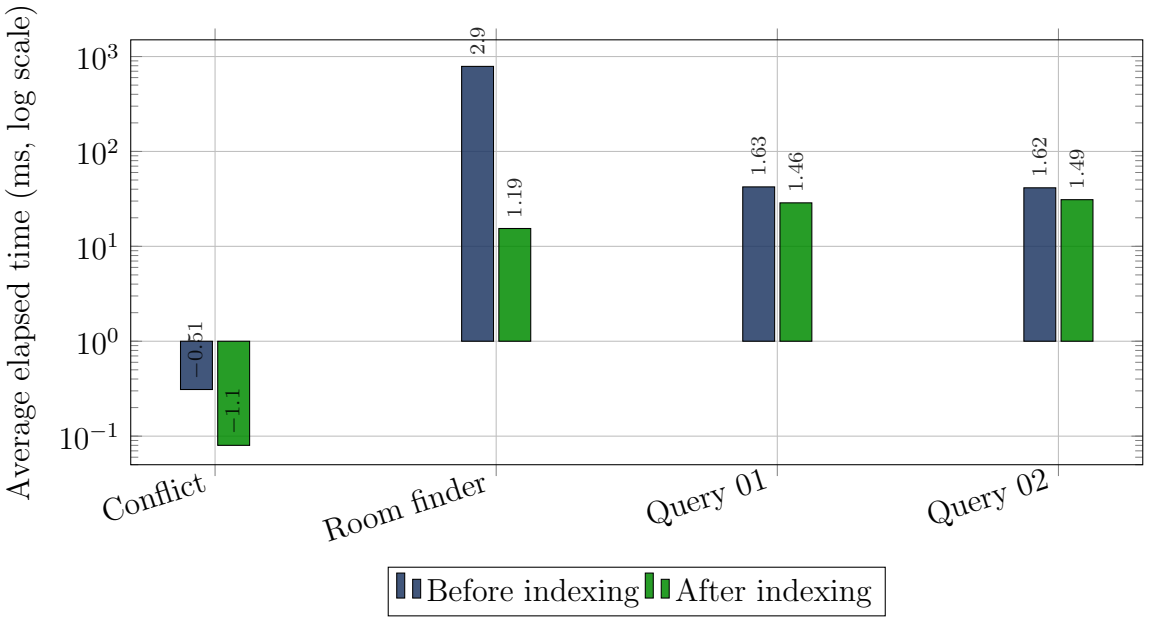

Figure 6: Elapsed-time comparison for the four tuned workloads.

§5.4 Execution-Plan Interpretation

Target	Before	After
Conflict check	Clustered request scan and decision lookup	Selective request and approved-decision seeks
Room finder	Repeated clustered scans of requests, facilities, and maintenance	Seeks on all repeated large/correlated accesses; the inexpensive 62-row space scan remains
Query 01	Clustered scans of both large booking relations	Semester range seek on requests and narrow approved-only decision scan
Query 02	Clustered scans of decisions and requests	Approved decision-time range seek and narrower covered request scan

Table 13: Important execution-plan changes.

The strongest change is the room finder: average elapsed time falls from 787.54 ms to 15.42 ms, while logical reads fall by 97.29%. Query 02 retains a residual `DATEPART(HOUR, decision_time)` expression, so its indexing gain is limited by grouping and computation after the semester range has been selected.

§5.5 Trade-offs and Recommendation

All six indexes were selected by SQL Server during measured runs. Together they use approximately 19.07 MB. The two request indexes account for most of the space and increase booking-write maintenance, but their key orders serve different access patterns: same-space interval checks versus semester ranges.

SQL Server also suggested a standalone request-status index. It was rejected because the condition `status <> 'cancelled'` qualifies roughly 93,029 of 100,030 requests and

is therefore not selective. Treating missing-index output as evidence rather than an automatic instruction avoids storage and write overhead with little expected benefit.
The final recommendation is to retain the six measured workload-specific indexes, monitor their write cost and fragmentation, and reconsider them only if booking volume or report patterns change materially.

§6 Functional Dependencies and Normalization

This section analyzes the functional dependencies (FDs) of the relational schema and verifies whether the database satisfies the Third Normal Form (3NF). The analysis is based on the primary keys, foreign keys, and attribute dependencies represented in the ERD.

§6.1 Functional Dependencies

A functional dependency $X \rightarrow Y$ means that the values of attributes X uniquely determine the values of attributes Y.
For this database, the functional dependencies are derived from the candidate keys and the semantic meaning of each entity. Foreign keys alone do not introduce additional functional dependencies in a relation when the attributes determined by the referenced relation are not stored in the child relation.
The following table summarizes the main functional dependencies.

Table 14: Functional Dependencies of the Relational Schema

Relation	Candidate Key	Functional Dependency
SPACE_TYPE	space_type_id	space_type_id → space_type_name
ROLE	role_id	role_id → role_name
SPACE	space_code	space_code → space_type_id, space_name, building, floor, room_number, capacity, current_status
USERS	user_id	user_id → role_id, full_name, email, phone_number, department, account_status
FACILITY	facility_id	facility_id → space_code, facility_name, description
BOOKING_REQUEST	booking_id	booking_id → user_id, space_code, start_time, end_time, purpose, expected_participants, booking_type, status
BOOKING_DECISION	decision_id	decision_id → booking_id, is_approved, is_automatic, decided_by_staff, decision_reason, decision_time
MAINTENANCE_RECORD	maintenance_id	maintenance_id → space_code, report_user, assigned_staff, problem_description, start_time, end_time, status, result_note, impact_level
USAGE_SESSION	session_id	session_id → decision_id, checked_in_by_staff, completed_by_staff, start_time, end_time, initial_condition, final_condition, usage_note
BOOKING_NOTIFICATION	(booking_id, maintenance_id, notification_type)	(booking_id, maintenance_id, notification_type) → notification_time
AUTO_USAGE_POLICY	(space_type_id, role_id)	No non-trivial FD involving non-key attributes.

§6.2 Important Observation about Foreign Keys

Foreign keys do not automatically create functional dependencies from the foreign key to the attributes of the referenced relation.
For example, the following dependency exists in the SPACE_TYPE relation:

$$\texttt{space\_type\_id} \rightarrow \texttt{space\_type\_name}.$$

However, `space_type_id` in `SPACE` does not determine `space_type_name` because `space_type_name` is not an attribute of `SPACE`. It is stored in the `SPACE_TYPE` relation.
Similarly,

$$\texttt{role\_id} \rightarrow \texttt{role\_name}$$

holds in `ROLE`, but it does not create a transitive dependency inside `USERS`, because `role_name` is not stored in `USERS`.
This separation prevents unnecessary redundancy between related entities.

§6.3 First Normal Form (1NF)

A relation is in First Normal Form if:

1. every attribute contains atomic values;
2. there are no repeating groups; and
3. each tuple can be uniquely identified by a key.

All relations in the proposed schema satisfy these requirements. Each attribute represents a single value, and each relation has a defined primary key.
For example, `BOOKING_REQUEST` stores one value for `start_time`, one value for `end_time`, one value for `purpose`, and so on. No attribute contains a repeating group of values.
Therefore,

All relations are in 1NF.

§6.4 Second Normal Form (2NF)

A relation is in Second Normal Form if:

1. it is already in 1NF; and
2. every non-prime attribute is fully functionally dependent on the whole candidate key.

Most relations in the proposed schema have a single-attribute candidate key, such as:

$$\texttt{space\_code}, \quad \texttt{user\_id}, \quad \texttt{booking\_id}, \quad \texttt{decision\_id},$$
$$\texttt{maintenance\_id}, \quad \texttt{session\_id}.$$

For a single-attribute key, a partial dependency cannot occur because there is no proper subset of the key.
The two relations with composite keys are `BOOKING_NOTIFICATION` and `AUTO_USAGE_POLICY`.
For `BOOKING_NOTIFICATION`:

$$(\texttt{booking\_id}, \texttt{maintenance\_id}, \texttt{notification\_type}) \rightarrow \texttt{notification\_time}.$$

The only non-key attribute, `notification_time`, depends on the complete composite key. There is no FD from a proper subset of the key to `notification_time` specified by the ERD.
For `AUTO_USAGE_POLICY`, the relation contains only its composite primary key and no non-key attributes. Therefore, partial dependency is impossible.
Consequently,

All relations are in 2NF.

§6.5 Third Normal Form (3NF)

A relation is in Third Normal Form if, for every non-trivial functional dependency

$$X \to A,$$

at least one of the following conditions holds:

1. X is a superkey; or
2. A is a prime attribute.

The schema satisfies the stronger condition that every determinant of a non-trivial FD is a candidate key.

§6.5.1 Verification of Individual Relations

`SPACE_TYPE` The only non-trivial FD is:

$$\texttt{space\_type\_id} \to \texttt{space\_type\_name}.$$

Since `space_type_id` is the primary key, it is a superkey.
Therefore, `SPACE_TYPE` is in 3NF.

`ROLE`

$$\texttt{role\_id} \to \texttt{role\_name}.$$

Since `role_id` is the primary key, the determinant is a superkey. Therefore, `ROLE` is in 3NF.

`SPACE`

$$\begin{aligned}\texttt{space\_code} \to \{&\texttt{space\_type\_id}, \texttt{space\_name}, \texttt{building},\\ &\texttt{floor}, \texttt{room\_number}, \texttt{capacity},\\ &\texttt{current\_status}\}.\end{aligned}$$

The determinant `space_code` is the primary key. Therefore, all non-trivial dependencies satisfy the 3NF condition.

`USERS`

$$\begin{aligned}\texttt{user\_id} \to \{&\texttt{role\_id}, \texttt{full\_name}, \texttt{email},\\ &\texttt{phone\_number}, \texttt{department},\\ &\texttt{account\_status}\}.\end{aligned}$$

The determinant is the primary key `user_id`. Therefore, `USERS` is in 3NF.

FACILITY

$$\texttt{facility\_id} \to \{\texttt{space\_code},\ \texttt{facility\_name},\ \texttt{description}\}.$$

Since facility_id is the primary key, the relation satisfies 3NF.

BOOKING_REQUEST

$$\texttt{booking\_id} \to \{\texttt{user\_id},\ \texttt{space\_code},\ \texttt{start\_time},\ \texttt{end\_time},\ \texttt{purpose},\ \texttt{expected\_participants},\ \texttt{booking\_type},\ \texttt{status}\}.$$

The determinant is the primary key booking_id. Therefore, BOOKING_REQUEST is in 3NF.

BOOKING_DECISION

$$\texttt{decision\_id} \to \{\texttt{booking\_id},\ \texttt{is\_approved},\ \texttt{is\_automatic},\ \texttt{decided\_by\_staff},\ \texttt{decision\_reason},\ \texttt{decision\_time}\}.$$

The determinant decision_id is the primary key. Therefore, the relation satisfies 3NF.

MAINTENANCE_RECORD

$$\texttt{maintenance\_id} \to \{\texttt{space\_code},\ \texttt{report\_user},\ \texttt{assigned\_staff},\ \texttt{problem\_description},\ \texttt{start\_time},\ \texttt{end\_time},\ \texttt{status},\ \texttt{result\_note},\ \texttt{impact\_level}\}.$$

The determinant is the primary key maintenance_id. Therefore, MAINTENANCE_RECORD is in 3NF.

USAGE_SESSION

$$\texttt{session\_id} \to \{\texttt{decision\_id},\ \texttt{checked\_in\_by\_staff},\ \texttt{completed\_by\_staff},\ \texttt{start\_time},\ \texttt{end\_time},\ \texttt{initial\_condition},\ \texttt{final\_condition},\ \texttt{usage\_note}\}.$$

The determinant session_id is the primary key. Therefore, USAGE_SESSION is in 3NF.

BOOKING_NOTIFICATION The candidate key is:

$$K = (\texttt{booking\_id}, \texttt{maintenance\_id}, \texttt{notification\_type}).$$

The non-trivial FD is:

$$K \to \texttt{notification\_time}.$$

The determinant is the complete candidate key and is therefore a superkey. Hence, BOOKING_NOTIFICATION satisfies 3NF.

`AUTO_USAGE_POLICY` The candidate key is:

$$K = (\texttt{space\_type\_id}, \texttt{role\_id}).$$

The relation contains no non-key attributes. Therefore, there is no non-trivial FD whose right-hand side is a non-prime attribute. Consequently, the relation trivially satisfies 3NF.

§6.6 Summary of Normalization Analysis

Table 15 summarizes the normalization status of all relations.

Table 15: Normalization Summary

Relation	1NF	2NF	3NF	Reason
SPACE_TYPE	Yes	Yes	Yes	Key determines all attributes
ROLE	Yes	Yes	Yes	Key determines all attributes
SPACE	Yes	Yes	Yes	Single-attribute key
USERS	Yes	Yes	Yes	Single-attribute key
FACILITY	Yes	Yes	Yes	Single-attribute key
BOOKING_REQUEST	Yes	Yes	Yes	Single-attribute key
BOOKING_DECISION	Yes	Yes	Yes	Single-attribute key
MAINTENANCE_RECORD	Yes	Yes	Yes	Single-attribute key
USAGE_SESSION	Yes	Yes	Yes	Single-attribute key
BOOKING_NOTIFICATION	Yes	Yes	Yes	Full composite-key dependency
AUTO_USAGE_POLICY	Yes	Yes	Yes	No non-key attributes

§6.7 Proof of 3NF

For every relation in the proposed database, the non-trivial functional dependencies identified from the ERD have a determinant that is a candidate key and therefore a superkey.
Formally, for every non-trivial FD

$$X \to A,$$

in the schema,

$$X \text{ is a superkey.}$$

Therefore, the 3NF condition is satisfied for every relation:

$$\boxed{\forall X \to A, \quad X \text{ is a superkey}}$$

Thus,

$$\boxed{\text{The complete relational schema satisfies 3NF.}}$$

In fact, under the functional dependencies identified above, the schema satisfies the stronger Boyce–Codd Normal Form (BCNF), because every determinant of a non-trivial functional dependency is a candidate key.

$$\boxed{\text{3NF is satisfied, and the relations are also in BCNF.}}$$

§6.8 Need for Further Decomposition

No further decomposition is required to achieve 3NF because the current relational schema already satisfies 3NF.
The decomposition of descriptive information into separate relations such as `ROLE`, `SPACE_TYPE`, and `SPACE` also prevents redundant storage of attributes that belong to different entities. For example, the role name is stored only in `ROLE`, while `USERS` stores only the corresponding foreign key `role_id`.
Therefore, the final schema preserves the entity structure represented by the ERD while avoiding partial and transitive dependencies within the individual relations.

§7 LLM Model and Agent Improvement Process

The experiment agents were run with **DeepSeek V4 Flash** through the **OpenCode** framework. The model was used to produce benchmark artifacts for evaluation and knowledge refinement; the final Phase 2 design, SQL, experiments, and report were reviewed and finalized by the project members.
Agent improvement followed a human-supervised, three-round cycle for each project section. In each round, the current Global Agent knowledge and the relevant section-specific Skill were used to generate a benchmark. The result was scored against a fixed evaluation rubric, weaknesses and reusable lessons were recorded, and those lessons were incorporated into the Skill and, when generally applicable, the Global Agent before the next round. This process improved requirement traceability, SQL Server correctness, consistency across artifacts, and resistance to unsupported assumptions while keeping final-submission decisions under human control.

§8 Conclusion

Phase 2 successfully extends the Campus Space Management System for the operating conditions observed during the pilot. The schema distinguishes advisory maintenance from out-of-service work, preserves maintenance-related booking notifications, normalizes roles and space types, unifies automatic and staff decisions, and migrates Phase 1 records without discarding their history. Four operational triggers process new requests, create maintenance notifications, protect decided schedules, and enforce usage-session eligibility.
The concurrency experiment demonstrates the central correctness risk and its prevention. The unsafe schedule produces two overlapping approvals, whereas the protected `BOOKING_REQUEST`–then–`SPACES` update-lock protocol leaves exactly one approved request and no overlapping approved pair. Automatic and staff approvals use this stored-procedure path; direct decision writes are not part of the protected workflow.
The deterministic generator supplies three academic years and 100,000 new booking records. The four required reports cover utilization hours, weekday/hour distribution, suitable-space discovery, and maintenance-escalation impact. Six measured indexes improve every tuned workload; most notably, the room finder reduces logical reads by 97.29% and elapsed time by 98.04%.
Finally, the functional-dependency analysis shows that all eleven relations satisfy at least Third Normal Form. The resulting design provides a consistent basis for concurrent

operations, operational reporting, and future growth while retaining clear trade-offs between read performance and index-maintenance cost.